DZIEN 3

MODUL 5 — OPERACJE NA PLIKACH I FOLDERACH

Zadanie 1 — Kopiowanie plikow

Temat: Operacje na plikach + For Each File in Folder + Path Exists

OPIS ZADANIA

1. Skopiuj pliki z rozszerzeniem `.xlsx` do folderu wybranego przez uzytkownika
2. Sprawdz czy skopiowany plik istnieje w folderze docelowym
3. Wyswietl informacje w logu: ktory plik jest procesowany i czy kopiowanie sie powiodlo

WSKAZOWKI

- Do pobrania biezacego folderu uzyj `Get Environment Variable` (zmienna: `CurrentDirectory`)
- Do wyboru folderu docelowego uzyj `Browse For Folder` (Select Folder)
- Uzyj `For Each File in Folder` z filtrem `*.xlsx`
- Do kopiowania uzyj `Copy File`
- Do sprawdzenia istnienia pliku uzyj `Get Local File or Folder` (PathType: File)
- Sciezke docelowa buduj przez `Path.Combine( str_docelowy_folder, CurrentFile.Name)`

Zadanie 2 — Rozpakowanie i raport plikow

Temat: Extract/Unzip + Build Data Table + FileInfo + Write CSV

OPIS ZADANIA

1. Rozpakuj plik `Raporty.zip` do folderu `Raporty`
2. Korzystajac z **DirectoryInfo** i **FileInfo** stworz tabele zawierajaca szczegoly rozpakowanych plikow:

Nazwa pliku	Folder	Rozszerzenie	Rozmiar
...dane pobrane z FileInfo...			

3. Zapisz te tabele jako plik CSV w folderze z rozpakowanymi plikami

WSKAZOWKI

- Uzyj `Extract/Unzip Files` z opcja **Extract to dedicated folder** — tworzy folder o nazwie pliku ZIP
- Do stworzenia tabeli uzyj `Build Data Table` z kolumnami: Nazwa pliku, Folder, Rozszerzenie, Rozmiar
- Iteruj po plikach za pomoca `For Each File in Folder`
- Uzyj `Modify Text` z operacja **Find and Replace** do usuniecia rozszerzenia z nazwy pliku (zastap `CurrentFile.Extension` pustym ciagiem)
- Rozszerzenie mozesz sformatowac do duzych liter (`ToUpper`)
- Dodaj wiersz do tabeli przez `Add Data Row` (ArrayRow)
- Na koniec uzyj `Write CSV` do zapisu tabeli

Zadanie 3 — Sortowanie plikow do folderow

Temat: Create Folder + Switch + Move File + For Each File in Folder

OPIS ZADANIA

1. W katalogu `Pliki do zadan` stworz foldery: **PDF**, **DOCX**, **XLSX**
2. Zawartosc folderu `Pliki do zadan` przeniesc do odpowiadajacych rozszerzeniu folderow
3. Poinformuj (Log Message) jakie pliki przenosisz i do jakiego folderu

WSKAZOWKI

- Nazwy folderow docelowych przechowuj w tablicy: `{"PDF", "DOCX", "XLSX"}`
- Uzyj `For Each` do tworzenia folderow przez `Create Folder`
- Do sortowania plikow uzyj `For Each File in Folder` i aktywnosci `Switch`
- Klucz Switch: `CurrentFile.Extension.Replace(".", Nothing).ToUpper`
- W kazdym case (PDF, DOCX, XLSX) uzyj `Move File` z odpowiednia sciezka docelowa
- Sciezke buduj przez `Path.Combine(str_folder_z_plikami, "PDF")`

ROZSZERZENIE Zadanie 3 — Podsumowanie przeniesionych plikow

Temat: Dictionary(Of String, Int32) jako licznik + zliczanie plikow wg rozszerzenia

OPIS ZADANIA

1. Po przeniesieniu plikow (Zadanie 3) wyswietl **podsumowanie**: ile plikow trafilo do kazdego folderu
2. Uzyj `Dictionary(Of String, Int32)` do zliczania plikow wg rozszerzenia
3. Na koniec wyswietl wynik w logu: np. `"PDF: 3 pliki, DOCX: 2 pliki"`

WSKAZOWKI

- Zainicjalizuj: `dict_podsumowanie = New Dictionary(Of String, Int32)`
- W kazdym Case (Zadanie 3) dodaj zliczanie: `If dict_podsumowanie.ContainsKey("PDF") Then dict_podsumowanie("PDF") = dict_podsumowanie("PDF") + 1 Else dict_podsumowanie("PDF") = 1`
- Po petli: uzyj `For Each` po `dict_podsumowanie` i `Log Message`: `pair.Key + ": " + pair.Value.ToString + " plikow"`

DODATKOWE Zadanie 4 — Monitoring folderu z raportem email

Temat: For Each File in Folder + CreationTime + List(Of String) + Send SMTP Mail Message

OPIS ZADANIA

1. Monitoruj folder `Raporty` — jesli w ciagu ostatnich **24h** pojawil sie nowy plik `.xlsx`, wyslij email z lista nowych plikow
2. Jesli nie ma nowych plikow — wyswietl Log Message: `"Brak nowych plikow w ostatnich 24h"`

WSKAZOWKI

- Uzyj `Now.AddHours(-24)` jako date graniczna
- Iteruj po plikach `*.xlsx` w folderze Raporty
- Sprawdzaj: `CurrentFile.CreationTime > dat_granica`
- Dodawaj nazwy nowych plikow do `List(Of String)` przez `Add To Collection`
- Tresc emaila buduj przez: `String.Join(vbNewLine, lst_nowe_pliki)`
- Do wysylki uzyj `Send SMTP Mail Message`

MODUL 6 — FLOWCHART + BIALA LISTA

Zadanie — Sprawdzanie firm na Bialej Liscie VAT

Temat: Flowchart + automatyzacja przegladarki + For Each Row + Type Into

OPIS ZADANIA

1. Na Flowcharcie stworz workflow, ktory sprawdzi na podstawie numeru NIP, czy dana firma figuruje w rejestrze VAT (**Biala Lista**)
2. Adres strony: `https://www.podatki.gov.pl/wykaz-podatnikow-vat-wyszukiwarka`
3. Dane NIP pobierz z pliku `CEIDG.xlsx`
4. Dla kazdego wiersza z tabeli wpisz NIP w wyszukiwarke i sprawdz wynik

WSKAZOWKI

- Uzyj `Read Range` do wczytania pliku `CEIDG.xlsx` do DataTable
- Iteruj po wierszach przez `For Each Row in DataTable`
- Uzyj `Type Into` z wyrazeniem: `row_wiersz_tabeli.Item("NIP").ToString + "[k(Enter)]"` — wpisuje NIP i zatwierdza Enterem
- Uzyj `Use Application/Browser` z trybem Close: Never (aby nie zamykac przegladarki miedzy iteracjami)
- Do sprawdzenia wyniku uzyj `Check App State`:
  - **Target appears** (tekst "Podmiot czynny"): `str_status_VAT = "Czynny"`
  - **Target does not appear** ("Nie znaleziono"): `str_status_VAT = "Niezarejestrowany"`
- Zapisz wynik do pliku excel: `"F" + (int_licznik+1).ToString`

ROZSZERZENIE ZAPIS WYNIKOW DO EXCEL

1. Przed petla: `Add Data Column` — dodaj kolumne `"Status VAT"` do `dt_CEIDG`
2. W petli: zapisuj wynik do `dt_CEIDG.Rows(int_licznik - 1)("Status VAT") = str_status_VAT`
3. Po petli: `Write Range` — zapisz `dt_CEIDG` do `CEIDG_wyniki.xlsx`

MODUL 7 — ORCHESTRATOR

Zadanie 1 — Logowanie z uzyciem Assetow

Temat: Orchestrator Assets (Text + Credential) + Get Asset + Get Credential

OPIS ZADANIA

1. Stworz w Orchestratorze **Asset typu Text** ze sciezka do aplikacji `Robotivity.exe`
2. Stworz w Orchestratorze **Asset typu Credential** z poswiadczeniem `admin/password`
3. Pobierz te Assety do robota
4. Uruchom aplikacje Robotivity i zaloguj sie do niej za pomoca pobranych Assetow
5. Sprawdz czy logowanie sie powiodlo (`Check App State`) — jesli TAK, wyswietl Log Message; jesli NIE, kliknij OK i wyswietl blad

WSKAZOWKI

- Do pobrania sciezki uzyj `Get Asset` (typ Text)
- Do pobrania loginu i hasla uzyj `Get Credential` — haslo jest typu `SecureString`
- Przed logowaniem uzyj `Kill Process` (robotivity) — czyste srodowisko
- Do wpisania hasla uzyj `Type Into` z wlasciwoscia **SecureText** (nie Text!)
- Uzyj `Check App State` aby zweryfikowac logowanie (szukaj elementu z formularza po zalogowaniu, np. przycisk "Accept")

Zadanie 2 — Dispatcher (dodawanie transakcji do Queue)

Temat: Orchestrator Queues + Add Queue Item / Bulk Add Queue Items

OPIS ZADANIA

1. Stworz **Queue** w Orchestratorze (folder My Workspace)
2. Wczytaj dane z pliku `Transactions.xlsx`
3. Dodaj dane jako **Queue Items** do utworzonej kolejki
4. Dodaj Log Message z informacja o dodanych transakcjach

WSKAZOWKI

- Do hurtowego dodania uzyj `Bulk Add Queue Items`
- **BONUS** Dodaj transakcje po jednej (For Each Row + `Add Queue Item`) z unikatowa referencja (licznik)
- Pamietaj o ustawieniu **FolderPath** na swoj folder w Orchestrator

> **Uwaga:** Jesli dodajesz oryginalny numer referencji, scope zmiennej-licznika musi byc wiekszy niz petla — inaczej wartosci sie gubia.

Zadanie 3 — Performer (procesowanie Queue Items)

Temat: Flowchart + Get Transaction Item + Set Transaction Status + obsluga bledow

OPIS ZADANIA

1. Na **Flowcharcie** stworz workflow, ktory przeprocesuje Queue Items dodane w zadaniu 2
2. Dane do logowania do Robotivity pobierz z Asseta typu Credential
3. W przypadku zlego hasla zakoncz prace z komunikatem bledu systemowego `"Nie zalogowano"`
4. Pobieraj transakcje z kolejki (`Get Transaction Item`) w petli
5. Dla kazdej transakcji wpisz wartosci (CashIn, OnUsCheck, NotOnUsCheck) do formularza aplikacji Robotivity
6. Jesli wartosci **nie sa liczbowe**, zmien status transakcji na **Failed** (Error Type: `Business`)
7. Jesli wartosci sa poprawne, kliknij Accept i zmien status na **Successful**

WSKAZOWKI

- Uzyj `FlowDecision` do sprawdzenia: czy zalogowano? czy jest transakcja?
- Do pobrania transakcji uzyj `Get Transaction Item` (w Try-Catch — jesli brak, przypisz Nothing)
- Warunek kontynuacji: `TransactionItem IsNot Nothing`
- Do pobrania danych z QueueItem uzyj: `TransactionItem.SpecificContent.GetValueOrDefault("CashIn").ToString`
- Sprawdz numerycznosc: `CashIn.IsNumeric And OnUsCheck.IsNumeric And NotOnUsCheck.IsNumeric`
- Do ustawienia statusu uzyj `Set Transaction Status` (Successful lub Failed z ErrorType Business)
- Na koncu: `Kill Process` (robotivity) i Log Message o zakonczeniu

MODUL 8 — STATE MACHINE (WPROWADZENIE)

Zadanie — State Machine z Input Dialog

Temat: State Machine + Transitions + warunki + Final State

OPIS ZADANIA

1. Stworz robota opartego na **State Machine**
2. W stanie poczatkowym (Init) pobierz od uzytkownika **imie** (Input Dialog)
3. Jesli imie to `"Adam"` — przypisz nazwisko `"Nowak"` i przejdz do Final State
4. Jesli imie to `"Kamil"` — przypisz nazwisko `"Kowalski"` i przejdz do Final State
5. Dla innego imienia — wrocz do stanu Init (powtorz pytanie)
6. W Final State wyswietl Log Message: `"Twoja osoba to: " + str_imie`

WSKAZOWKI

- Stworz nowy projekt typu **State Machine** (nie Sequence/Flowchart)
- Dodaj dwa stany: `State` (Init) i `Final State`
- W stanie Init: dodaj `Input Dialog` w sekcji **Entry**
- Stworz trzy **Transitions** ze stanu Init:
  - Transition 1: Warunek `str_imie = "Adam"` → Action: Assign `str_nazwisko = "Nowak"` → Final State
  - Transition 2: Warunek `str_imie = "Kamil"` → Action: Assign `str_nazwisko = "Kowalski"` → Final State
  - Transition 3: Bez warunku (domyslna) → wraca do Init
- Final State: dodaj `Log Message` w sekcji **Entry**

> **Klucz:** Transitions sa ewaluowane w kolejnosci — domyslna (bez warunku) musi byc ostatnia, inaczej zawsze sie wykona!

DZIEN 4

MODUL 9 — DICTIONARY

Zadanie — Kursy walut w Dictionary

Temat: Data Scraping + Dictionary<String, String> + For Each Row

OPIS ZADANIA

1. Otworz strone `https://nbp.pl/statystyka-i-sprawozdawczosc/kursy/tabela-a/`
2. Zaczytaj dane z tabeli kursow walut i stworz plik `Waluty.xlsx`
3. Przypisz wartosci z DataTable do `Dictionary(Of String, String)` — klucz: **kod waluty**, wartosc: **kurs sredni**
4. Wyswietl kurs Euro w Log Message: `"Kurs euro to: " + dict_waluty("1 EUR").ToString`

WSKAZOWKI

- Uzyj `Use Application/Browser` z Chrome
- Do pobrania tabeli uzyj `Extract Table Data`
- Zapisz tabele przez `Write Range Workbook` do `Waluty.xlsx`
- Zainicjalizuj slownik: `New Dictionary(Of String, String)`
- W petli `For Each Row in DataTable` przypisuj wartosci:

```
klucz = CurrentRow.Item(1).ToString
wartosc = CurrentRow.Item(2).ToString
dict_waluty(klucz) = wartosc
```

- Uzyj `Multiple Assign` do jednoczesnego przypisania klucza i wartosci

DODATKOWE Zadanie 2 — Tlumacz walut (interaktywny)

Temat: Dictionary + ContainsKey + While + Input Dialog + walidacja danych

OPIS ZADANIA

1. Wykonaj kroki 1-4 z Zadania Dictionary (pobierz tabele NBP → Dictionary)
2. Uzytkownik wpisuje kod waluty (np. `"1 EUR"`, `"1 USD"`) przez **Input Dialog**
3. Robot zwraca kurs z Dictionary w **Message Box**
4. Jesli kod nie istnieje — poinformuj uzytkownika i ponow pytanie
5. Wpisanie `"koniec"` zamyka robota

WSKAZOWKI

- Uzyj petli `While` z flaga `bl_kontynuuj = True`
- Warunek wyjscia: `str_kod_waluty.ToLower = "koniec"` → `bl_kontynuuj = False`
- Przed odczytem ze slownika sprawdz: `dict_waluty.ContainsKey(str_kod_waluty)`
- Jesli klucz istnieje: `"Kurs " + str_kod_waluty + " wynosi: " + dict_waluty(str_kod_waluty)`
- Jesli nie: `"Nie znaleziono waluty: " + str_kod_waluty + ". Sprobuj ponownie."`

> **Klucz:** `dict.ContainsKey()` zapobiega `KeyNotFoundException` — zawsze sprawdzaj przed dostepem do slownika!

MODUL 10 — ROZBUDOWANE STATE MACHINE

Zadanie — State Machine z obsluga wyjatkow

Temat: State Machine + Try-Catch + Business/System Exception + Robotivity

OPIS ZADANIA

1. Uzywajac **State Machine** stworz robota, ktory:
   - Otworzy aplikacje **Robotivity** (w przypadku bledu robot ma zakonczyc dzialanie)
   - Wprowadzi transakcje z pliku `Transactions.xlsx`
   - Na koncu wyswietli Log Message o zakonczeniu pracy
2. Obsluz **Business Exception** — jesli dane nie sa numeryczne, pomin transakcje i przejdz do nastepnej
3. Obsluz **System Exception** — w przypadku bledu systemowego, restartuj aplikacje i wznow prace

STANY STATE MACHINE

Stan	Opis
Init	Zbierz zmienne, wyczysc srodowisko (Kill Process), otworz aplikacje, pobierz transakcje (Read Range), Try-Catch
Get Transaction	Sprawdz czy jest dostepna transakcja (wiersz z DT). Jesli brak → Final State. Korzystaj ze zmiennej `TransactionNumber` (default = 1)
Process	Przetwarzaj transakcje. Business Exception → pomin, System Exception → restart. Po tranzycji wyczysc zmienna Business Rule Exception
Final State	Zamknij aplikacje, wyswietl podsumowanie

WSKAZOWKI

- Uzyj zmiennej `TransactionNumber` (Int32, default: 1) do sledzenia biezacej transakcji
- Pobieraj wiersz: `dt_transakcje.Rows(TransactionNumber - 1)`
- Warunek braku transakcji: `TransactionNumber > dt_transakcje.Rows.Count`
- W stanie Process: Try-Catch — Catch `BusinessRuleException` (pomin) i `System.Exception` (restart)
- Po kazdej transakcji inkrementuj: `TransactionNumber = TransactionNumber + 1`

ROZSZERZENIE State Machine — Dodatkowy State "Raport"

Temat: Build Data Table + Write Range + dodawanie nowego State do State Machine

OPIS ZADANIA

1. Dodaj nowy stan **"State - Raport"** miedzy Procesowaniem a Final State
2. W stanie Raport wygeneruj podsumowanie przetworzonych transakcji
3. Zapisz raport do pliku `Raport_transakcji.xlsx`

WSKAZOWKI

- Uzyj `Build Data Table` z kolumnami: `Nr transakcji`, `Status`, `Szczegoly`
- Wypelnij tabele na podstawie zebranych danych (zliczaj sukcesy/porazki w procesowaniu)
- Uzyj `Write Range` do zapisu do `Raport_transakcji.xlsx`
- Zmodyfikuj Transitions:
  - Get Transaction: `row_TransactionItem Is Nothing` → **State - Raport** (zamiast Final State)
  - State - Raport: (bezwarunkowa) → **Final State**

MODUL 11 — REFRAMEWORK

Cwiczenie wprowadzajace — REFramework

Temat: Omowienie struktury Robotic Enterprise Framework (wspolnie z prowadzacym)

OPIS CWICZENIA

1. Zapoznaj sie ze struktura szablonu **REFramework** (Robotic Enterprise Framework)
2. Przeanalizuj pliki w folderach: `Framework/` , `Data/` , `Tests/`
3. Omow role poszczegolnych plikow:

Plik	Rola
`Main.xaml`	Glowna State Machine (Init, Get Transaction, Process, End Process)
`InitAllSettings.xaml`	Wczytuje konfiguracje z Config.xlsx
`InitAllApplications.xaml`	Otwiera wymagane aplikacje
`GetTransactionData.xaml`	Pobiera kolejna transakcje (z Queue lub pliku)
`Process.xaml`	Przetwarza pojedyncza transakcje
`SetTransactionStatus.xaml`	Ustawia status transakcji (Success/Business/System)
`CloseAllApplications.xaml`	Zamyka aplikacje
`KillAllProcesses.xaml`	Awaryjne zamkniecie procesow
`RetryCurrentTransaction.xaml`	Logika ponawiania transakcji
`TakeScreenshot.xaml`	Screenshot przy bledzie
`Data/Config.xlsx`	Konfiguracja: sciezki, nazwy assetow, kolejki, ustawienia
`Data/Input/`	Dane wejsciowe
`Data/Output/`	Dane wyjsciowe
`Data/Temp/`	Pliki tymczasowe
`Tests/`	Testy jednostkowe

KLUCZOWE POJECIA

- **Business Rule Exception** — blad danych (np. brak NIP) — transakcja jest pomijana
- **System Exception** — blad techniczny (np. crash aplikacji) — robot restartuje aplikacje i powtarza transakcje
- **Config.xlsx** — centralne miejsce konfiguracji (nie hardcoduj wartosci w workflow!)
- **MaxRetryNumber** — ile razy robot powtorzy transakcje po System Exception

CONFIG.XLSX — TRZY ARKUSZE

Arkusz	Opis	Przyklad
Settings	Ustawienia procesu (key-value)	OrchestratorQueueName, MaxRetryNumber
Constants	Stale wartosci	URL aplikacji, timeout
Assets	Nazwy assetow z Orchestratora	CredentialName, LogLevel

> **Uwaga techniczna:** Config w REFramework jest typu `Dictionary(Of String, Object)` — dlatego przy odczycie zawsze potrzebny jest `.ToString` : `Config("OrchestratorQueueName").ToString`

ALTERNATYWA BEZ ORCHESTRATORA — WINDOWS CREDENTIAL MANAGER

1. Panel sterowania → Menedzer poswiadczen → Poswiadczenia systemu Windows → Dodaj
2. W UiPath: `Get Secure Credential` zamiast `Get Credential`
   - Target: nazwa poswiadczenia z Credential Manager
   - Output: Username (String), Password (SecureString)

> **Uwaga:** Windows Credential Manager dziala tylko lokalnie. W produkcji zawsze uzywamy Orchestrator Assets.

MODUL 12 — DISPATCHER + REFRAMEWORK PERFORMER

Zadanie 4a — Dispatcher

Temat: Read Range + Bulk Add Queue Items + Orchestrator Queue

OPIS ZADANIA

1. Stworz robota (Dispatcher), ktorego zadaniem bedzie dodanie Queue Items z pliku `Transactions.xlsx` do Queue o nazwie `4a_Dispatcher`
2. Najpierw stworz Queue w Orchestratorze (folder My Workspace)
3. Dodaj Log Message z informacja o dodanych danych

WSKAZOWKI

- Przed wczytaniem pliku uzyj `Kill Process` (excel) — aby plik nie byl zablokowany
- Wczytaj dane przez `Read Range Workbook`
- Uzyj `Bulk Add Queue Items` z odpowiednia nazwa Queue i FolderPath

Zadanie 4b — REFramework Performer

Temat: REFramework + Get Transaction Item + Process.xaml + obsluga wyjatkow

OPIS ZADANIA

1. Stworz robota opartego na szablonie **REFramework**, ktory bedzie **Performerem** dla kolejki z zadania 4a
2. Robot ma:
   - Pobierac transakcje z kolejki `4a_Dispatcher`
   - Logowac sie do aplikacji Robotivity (dane z Orchestrator Assets)
   - Wprowadzac dane z transakcji (CashIn, OnUsCheck, NotOnUsCheck) do formularza
   - Obslugiwac **Business Exception** (dane nie sa liczbowe — pomin transakcje)
   - Obslugiwac **System Exception** (blad aplikacji — restart i powtorz)

WSKAZOWKI

- Uzyj szablonu REFramework z UiPath (File → New → REFramework)
- Skonfiguruj `Config.xlsx` : nazwa kolejki, sciezka do aplikacji, dane logowania
- W `InitAllApplications.xaml` : otworz Robotivity i zaloguj
- W `GetTransactionData.xaml` : `Get Transaction Item` z kolejki
- W `Process.xaml` : wpisz dane z transakcji do formularza i kliknij Accept
- W `SetTransactionStatus.xaml` : ustaw status na Successful lub Failed
- W `CloseAllApplications.xaml` : zamknij Robotivity
- W `KillAllProcesses.xaml` : `Kill Process` robotivity

> **Klucz:** REFramework to gotowa State Machine z obsluga bledow — nie musisz budowac jej od zera. Skup sie na dostosowaniu plikow Framework/ do swojego procesu.

ZADANIA BONUSOWE (JESLI ZOSTANIE CZAS)

BONUS Dashboard transakcji

Temat: Build Data Table + Add Data Row + Write Range + zliczanie statystyk

OPIS ZADANIA

1. Po przetworzeniu wszystkich transakcji (Performer), stworz raport z podsumowaniem: ile **Successful**, ile **Failed (Business)**
2. Na poczatku procesu: `int_successful = 0` , `int_failed_business = 0`
3. Po kazdej transakcji inkrementuj odpowiedni licznik
4. Na koncu: `Build Data Table` (kolumny: Status, Liczba) + `Add Data Row` + `Write Range` do `Dashboard_transakcji.xlsx`

BONUS Logowanie wynikow do Excela (REFramework)

Temat: SetTransactionStatus.xaml + Append Range + modyfikacja plikow Framework

OPIS ZADANIA

1. Zmodyfikuj `SetTransactionStatus.xaml` — po ustawieniu statusu zapisz wynik do pliku Excel
2. Loguj: **Reference**, **Status**, **Timestamp** do `Data/Output/TransactionLog.xlsx`

WSKAZOWKI

- `str_status = If(in_BusinessException Is Nothing, "Successful", "Failed: " + in_BusinessException.Message)`
- `str_reference = If(in_TransactionItem IsNot Nothing, in_TransactionItem.Reference, "N/A")`
- Uzyj `Append Range` (dopisuje bez nadpisywania) zamiast `Write Range`
- Wiersz: `{str_reference, str_status, Now.ToString("dd-MM-yyyy HH:mm:ss")}`